

Lezione 1: Basi di Java

Ingegneria del Software - Sede di Cremona

a.a. 2026/2027

La prima lezione introduce le basi del linguaggio **Java** partendo dalle strutture fondamentali: le variabili, i tipi primitivi, i cicli, l'input da tastiera, e le strutture dati (array e liste).

1 Variabili, input e cicli

Il primo esempio mostra come dichiarare e utilizzare variabili di tipo primitivo in Java, come `int`, `double` e `boolean`. In Java, una variabile di tipo `T` viene dichiarata e inizializzata a `V` con:
`T x = V;`

Le variabili di tipo primitivo sono sempre modificate per copia.

Tabella 1: Operatori aritmetici di base.

Operatore	Significato
+	Addizione
-	Sottrazione
*	Moltiplicazione
/	Divisione
%	Modulo (resto della divisione)

In Java, variabili di tipo `String` rappresentano sequenze di caratteri. E' possibile dichiarare e inizializzare una variabile di tipo `String` come:

```
String s = "stringa di prova";
```

Anche le stringhe sono modificate per copia.

Tabella 2: (Alcuni) Metodi utili per le stringhe.

Metodo	Descrizione	Esempio
<code>equals(String s)</code>	Confronta due stringhe per uguaglianza	<code>boolean uguali = s1.equals(s2)</code>
<code>equalsIgnoreCase(String s)</code>	Confronta due stringhe ignorando maiuscole/minuscole	<code>s1.equalsIgnoreCase(s2)</code>
<code>length()</code>	Restituisce la lunghezza della stringa	<code>int l = s.length()</code>
<code>isEmpty()</code>	Verifica se la stringa è vuota	<code>boolean vuota = s.isEmpty()</code>
<code>toUpperCase()</code>	Converte la stringa in maiuscolo	<code>s = s.toUpperCase()</code>
<code>toLowerCase()</code>	Converte la stringa in minuscolo	<code>s = s.toLowerCase()</code>
<code>charAt(int i)</code>	Restituisce il carattere in posizione <code>i</code>	<code>char c = s.charAt(2)</code>
<code>substring(int start, int end)</code>	Estrae una sottostringa	<code>String sub = s.substring(2, 10)</code>
<code>indexOf(String sub)</code>	Restituisce l'indice della prima occorrenza	<code>int i = s.indexOf("a")</code>
<code>lastIndexOf(String sub)</code>	Restituisce l'indice dell'ultima occorrenza	<code>int i = s.lastIndexOf("a")</code>
<code>contains(CharSequence cs)</code>	Verifica se la stringa contiene la sequenza	<code>s.contains("ciao")</code>
<code>startsWith(String prefix)</code>	Verifica se la stringa inizia con un prefisso	<code>s.startsWith("pre")</code>
<code>endsWith(String suffix)</code>	Verifica se la stringa termina con un suffisso	<code>s.endsWith("txt")</code>
<code>replace(char old, char new)</code>	Sostituisce tutti i caratteri indicati	<code>s.replace('a', 'b')</code>
<code>split(String regex)</code>	Divide la stringa in un array	<code>String[] parts = s.split(" ")</code>
<code>join(CharSequence sep, ...)</code>	Unisce più stringhe con un separatore	<code>String.join("-", "a", "b", "c")</code>
<code>format(String fmt, ...)</code>	Crea stringhe formattate	<code>String.format("Ciao %s", nome)</code>

L'istruzione:

```
System.out.println(s);
```

stampa la stringa `s` in console.

Inoltre, leggere dati inseriti dall'utente da tastiera è possibile grazie alla classe `Scanner`.

```
1 import java.util.Scanner;
2
3 // Dichiarazione stringa (senza inizializzazione)
4 String input_utente;
5 // Dichiarazione scanner per leggere da console
6 Scanner scanner = new Scanner(System.in);
7 // Legge da console
8 input_utente = scanner.nextLine();
```

Listing 1: Lettura di input da tastiera.

I costrutti di base (statement condizionali e cicli) sono:

```
1 // Statement condizionale.
2 if (condizione 1) {
3     ...
4 } else if (condizione 2) {
5     ...
6 } else {
7     ...
8 }
9 // Ciclo while.
10 while (condizione di permanenza) {
11     ...
12 }
```

Listing 2: Costrutti di base in Java.

Esempio Completo: https://github.com/LesLivia/IngSw_2627/blob/main/src/main/warm_up/Step1.java

2 Array e liste

Il secondo esempio introduce le strutture dati di base (array e liste) e i cicli `for` che, come in altri linguaggi di programmazione, necessitano di una variabile che funga da iteratore. In Java, data una struttura dati (non fa differenza che sia un array o una lista) è possibile anche utilizzare una versione compatta del ciclo `for` dichiarato come:

`for(T iteratore: struttura)`

che cicla direttamente sugli elementi della struttura senza passare dall'indice.

2.1 Array

Un **array** è una sequenza di elementi di dimensione fissa. Una volta inizializzato l'array, è possibile modificare il valore dei suoi elementi solo entro le dimensioni dichiarate. Non è, invece, possibile, aggiungere nuovi elementi all'array nè rimuoverli.

```
1 // Dichiarazione + inizializzazione inline
2 int[] numeri = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
3 System.out.println("numeri[2]: " + numeri[2]);
4 // Dichiarazione + assegnazione separata
5 int[] numeri2;
6 numeri2 = new int[10]; // di default assegna 0 a tutti
7 numeri2[2] = 10;
8 System.out.println("numeri2[2]: " + numeri2[2]);
9 // Dichiarazione + inizializzazione con 'new' inline
10 int[] numeri3 = new int[]{10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
11 System.out.println("numeri3[2]: " + numeri3[2]);
```

Listing 3: Inizializzazione di array in Java

2.2 Liste

A differenza dell'array, una lista può crescere e ridursi durante l'esecuzione, cioè è possibile aggiungere e rimuovere elementi. In Java, è possibile dichiarare una nuova lista come:

```
List<T> list = new ArrayList<T>();
```

dove T è il tipo selezionato per gli elementi della lista.

Sia gli array sia le stringhe sono sempre modificati per riferimento.

Tabella 3: Metodi utili per le liste in Java.

Metodo	Descrizione	Esempio
add(E e)	Aggiunge un elemento alla lista	lista.add(5);
add(int index, E e)	Inserisce un elemento in posizione index	lista.add(0, 10);
get(int index)	Restituisce l'elemento in posizione index	int x = lista.get(2);
getFirst()	Restituisce il primo elemento della lista	int x = lista.getFirst();
getLast()	Restituisce l'ultimo elemento della lista	int x = lista.getLast();
set(int index, E e)	Sostituisce l'elemento alla posizione indicata	lista.set(1, 99);
remove(int index)	Rimuove l'elemento alla posizione indicata	lista.remove(0);
remove(Object o)	Rimuove la prima occorrenza dell'oggetto	lista.remove(Integer.valueOf(5));
size()	Restituisce il numero di elementi nella lista	int n = lista.size();
isEmpty()	Verifica se la lista è vuota	if(lista.isEmpty()) { ... }
contains(Object o)	Verifica se la lista contiene l'elemento	if(lista.contains(3)) { ... }
clear()	Rimuove tutti gli elementi della lista	lista.clear();

Esempio Completo: https://github.com/LesLivia/IngSw_2627/blob/main/src/main/warm_up/Step2.java

Attività di rinforzo

1. Modificare il primo programma in modo che il ciclo termini non solo con la parola `esci`, ma anche se la variabile `x` diventa negativa.
2. Estendere il secondo programma aggiungendo un controllo: se l'utente inserisce un numero negativo, questo non deve essere aggiunto alla lista.
3. Alla fine del secondo programma, stampare la somma totale dei numeri inseriti.

3 PoliFlix: Registrazione Utente e Login

Il terzo esempio mostra una possibile implementazione di due funzionalità fondamentali di PoliFlix — la registrazione di un nuovo utente e il login — senza usare ancora la programmazione a oggetti. Tutto il codice risiede all'interno del metodo `main` o di funzioni di supporto, e i dati degli utenti vengono gestiti tramite variabili e liste.

Come salvare i dati relativi agli utenti che si registrano? Una possibilità è dichiarare liste separate per ogni feature, ad esempio:

```
1 List<String> utenti_username = new ArrayList<>();  
2 List<String> utenti_password = new ArrayList<>();
```

Domande chiave: cosa succederebbe se, per errore, il contenuto delle liste risultasse disallineato (ad esempio, 4 username e 3 password)? Cosa succederebbe se un utente provasse a registrarsi con uno username già esistente e quanto sarebbe macchinoso implementare tutti i controlli e le validazioni dei dati in input?

I dati relativi all'utente loggato al momento sono salvati in una lista di due elementi (username e password):

```
1 List<String> utente_loggato = new ArrayList<>();  
2 utente_loggato.add(username_inserito);  
3 utente_loggato.add(password_inserita);  
4  
5 System.out.println("Bentornato " + utente_loggato.get(0) + "!");
```

Domande chiave: Cosa succederebbe se uno sviluppatore esterno avesse necessità di accedere a tali informazioni? Cosa sarebbe necessario fare se volessimo aggiungere ulteriori informazioni sugli utenti, come email, livello di abbonamento o preferenze di contenuti?

In generale, concentrare la gestione delle funzionalità offerte all'utente nel main o in funzioni di supporto ne complica notevolmente lo sviluppo e la manutenzione. Immaginate, ad esempio, di avere utenti con diversi ruoli (per esempio, Admin, Standard, Premium), ognuno con accesso a funzionalità diverse.

Domanda chiave: Quanto sarebbe macchinoso aggiungere la gestione dinamica dell'interazione con l'utente in base al suo ruolo?

Esempio Completo: https://github.com/LesLivia/IngSw_2627/blob/main/src/main/warm_up/Step3_PoliFlix.java